

Guía Docente Diapositiva por Diapositiva — Sesión 2: Dockerfile Profesional, Capas, Optimización y Docker Hub

Curso: Docker desde Cero: Crea y Despliega Aplicaciones (10ma Edición 2026) **Instructor:** Ing. Cristian Jampier Chileno Segundo | OTI - UNI **Total Diapositivas:** 34

Instrucciones de Orientación Pedagógica

Esta guía contiene la explicación exacta y guión en primera persona para explicar **cada una de las diapositivas de la presentación oficial**. Úsala durante la clase virtual o presencial para garantizar que no quede ningún punto del temario sin abordar.

Explicación Diapositiva por Diapositiva

Diapositiva 1: DOCKER DESDE CERO:

Contenido de la PPT:

```
DOCKER DESDE CERO:  
Crea y Despliega Aplicaciones  
INSTRUCTOR: Cristian Jampier Chileno Segundo  
PROGRAMA DE  
INICIACIÓN  
TECNOLÓGICA  
PIT 2026  
Oficina de Tecnologías de la Información (OTI)  
Programa Completo ù PIT 2026
```

Guión del Docente (Lo que debes decir a los alumnos):

"Estimados estudiantes, en la **Diapositiva 1** abordamos un punto fundamental: *DOCKER DESDE CERO*. Revisen los conceptos clave presentados en pantalla y asegúrense de que los estudiantes comprendan cómo se conecta este punto con el laboratorio práctico de la sesión."

Acción en Consola / Pizarra:

```
# Demostración práctica recomendada para la Diapositiva 1  
docker build -t mi-flask:v1 .
```

Tip de Gestión del Aula:

- *Hacer una pregunta de pausa activa:* "¿Alguien tiene alguna duda sobre DOCKER DESDE CERO?"

Diapositiva 2: SESIÓN 2

Contenido de la PPT:

```
SESIÓN 2
1.Instrucciones Clave
2.Capas y Construcción
3.Optimización con Cachú
4.Uso de .dockerignore
5.Imágenes Base Livianas
6.Publicación en Docker Hub
```

Guión del Docente (Lo que debes decir a los alumnos):

"Estimados estudiantes, en la **Diapositiva 2** abordamos un punto fundamental: *SESIÓN 2*. Revisen los conceptos clave presentados en pantalla y asegúrense de que los estudiantes comprendan cómo se conecta este punto con el laboratorio práctico de la sesión."

Acción en Consola / Pizarra:

```
# Demostración práctica recomendada para la Diapositiva 2
docker build -t mi-flask:v1 .
```

Tip de Gestión del Aula:

- *Hacer una pregunta de pausa activa: "¿Alguien tiene alguna duda sobre SESIÓN 2?"*

Diapositiva 3: Escribir un Dockerfile con las instrucciones clave:

Contenido de la PPT:

```
Escribir un Dockerfile con las instrucciones clave:
FROM, RUN, COPY, ENV, EXPOSE y CMD.
Explicar cómo funcionan las capas y cómo afectan el
rendimiento del build.
Aprovechar la cachú de Docker para builds más rápidos.
Usar .dockerignore para evitar archivos innecesarios en
la imagen.
Elegir imágenes base livianas (slim, alpine) según el
caso.
Publicar imágenes en Docker Hub con tagging y
versionado.
Al terminar la sesión 2 podrás
De Dockerfile básico a imagen
profesional
Objetivo de la sesión 2
```

Gui3n del Docente (Lo que debes decir a los alumnos):

"Estimados estudiantes, en la **Diapositiva 3** abordamos un punto fundamental: *Escribir un Dockerfile con las instrucciones clave*:. Expliquen aqu3 el problema cl3sico en el desarrollo de software. En la laptop del desarrollador todo marcha perfecto porque tiene instaladas librer3as espec3ficas o variables locales, pero en el servidor de producci3n o en la PC del cliente falla. Docker soluciona esto empaquetando todo el entorno junto con la app."

Acci3n en Consola / Pizarra:

```
# Demostraci3n pr3ctica recomendada para la Diapositiva 3
docker build -t mi-flask:v1 .
```

Tip de gesti3n del Aula:

- *Hacer una pregunta de pausa activa*: "¿Alguien tiene alguna duda sobre Escribir un Dockerfile con las instrucciones clave:?"

Diapositiva 4: Recordatorio de la

Contenido de la PPT:

Recordatorio de la
sesi3n anterior

Gui3n del Docente (Lo que debes decir a los alumnos):

"Estimados estudiantes, en la **Diapositiva 4** abordamos un punto fundamental: *Recordatorio de la*. Revisen los conceptos clave presentados en pantalla y aseg3rense de que los estudiantes comprendan c3mo se conecta este punto con el laboratorio pr3ctico de la sesi3n."

Acci3n en Consola / Pizarra: *Escribir en la pizarra o proyectar el diagrama del flujo correspondiente.*

Tip de gesti3n del Aula:

- *Hacer una pregunta de pausa activa*: "¿Alguien tiene alguna duda sobre Recordatorio de la?"

Diapositiva 5: Contenerizaci3n: empaquetar app + dependencias.

Contenido de la PPT:

Contenerizaci3n: empaquetar app + dependencias.
Diferencia entre imagen (plantilla) y contenedor
(ejecuci3n).

Docker resuelve el problema de ¿en mi máquina funciona?.

Comandos esenciales: run, ps, stop, rm, images.

Construimos y ejecutamos una app Flask en contenedor.

Sabemos ejecutar contenedores desde imágenes existentes. Ahora crearemos nuestras propias imágenes de forma profesional.

Base para hoy

Lo que ya dominamos

Guión del Docente (Lo que debes decir a los alumnos):

"Estimados estudiantes, en la **Diapositiva 5** abordamos un punto fundamental: *Contenerización: empaquetar app + dependencias*.. Expliquen aquí el problema clásico en el desarrollo de software. En la laptop del desarrollador todo marcha perfecto porque tiene instaladas librerías específicas o variables locales, pero en el servidor de producción o en la PC del cliente falla. Docker soluciona esto empaquetando todo el entorno junto con la app."

Acción en Consola / Pizarra:

```
# Demostración práctica recomendada para la Diapositiva 5
docker build -t mi-flask:v1 .
```

Tip de Gestión del Aula:

- *Hacer una pregunta de pausa activa:* "¿Alguien tiene alguna duda sobre Contenerización: empaquetar app + dependencias.?"

Diapositiva 6: ¿QUÉ ES UN DOCKERFILE?

Contenido de la PPT:

¿QUÉ ES UN DOCKERFILE?

Un Dockerfile es un archivo de texto con instrucciones que Docker lee para construir una imagen automáticamente. Es la receta que define todo lo que necesita tu aplicación para ejecutarse.

Definición

Guión del Docente (Lo que debes decir a los alumnos):

"Estimados estudiantes, en la **Diapositiva 6** abordamos un punto fundamental: ¿QUÉ ES UN DOCKERFILE?. Expliquen que el Dockerfile es la receta de cocina. Cada instrucción (**FROM**, **WORKDIR**, **COPY**, **RUN**, **CMD**) construye una capa estandarizada e inmutable sobre la cual se monta nuestra aplicación."

Acción en Consola / Pizarra:

```
# Demostración práctica recomendada para la Diapositiva 6
docker build -t mi-flask:v1 .
```

💡 Tip de Gestión del Aula:

- *Hacer una pregunta de pausa activa:* "¿Alguien tiene alguna duda sobre `QUÉ ES UN DOCKERFILE??`?"

📄 Diapositiva 7: Instrucciones clave

Contenido de la PPT:

Instrucciones clave
del Dockerfile

👤 Guión del Docente (Lo que debes decir a los alumnos):

"Estimados estudiantes, en la **Diapositiva 7** abordamos un punto fundamental: *Instrucciones clave*. Expliquen que el Dockerfile es la receta de cocina. Cada instrucción (**FROM**, **WORKDIR**, **COPY**, **RUN**, **CMD**) construye una capa estandarizada e inmutable sobre la cual se monta nuestra aplicación."

🏠 Acción en Consola / Pizarra:

```
# Demostración práctica recomendada para la Diapositiva 7
docker build -t mi-flask:v1 .
```

💡 Tip de Gestión del Aula:

- *Hacer una pregunta de pausa activa:* "¿Alguien tiene alguna duda sobre Instrucciones clave?"

📄 Diapositiva 8: FROM: EL PUNTO DE PARTIDA

Contenido de la PPT:

FROM: EL PUNTO DE PARTIDA
1 FROM python :3.12 - slim
Evita :latest en producción. Fija una versión concreta (:3.12-slim) para builds reproducibles.
Versiones
Usa imágenes oficiales siempre que puedas. Son mantenidas, actualizadas y más seguras que imágenes de terceros.
Consejo
Define la imagen base sobre la que se

construye.
Es la primera instrucción de todo Dockerfile.
Puede ser una imagen oficial (python, node, nginx) o una imagen propia.

👤 Guión del Docente (Lo que debes decir a los alumnos):

"Estimados estudiantes, en la **Diapositiva 8** abordamos un punto fundamental: *FROM: EL PUNTO DE PARTIDA*. Expliquen que el Dockerfile es la receta de cocina. Cada instrucción (**FROM**, **WORKDIR**, **COPY**, **RUN**, **CMD**) construye una capa estandarizada e inmutable sobre la cual se monta nuestra aplicación."

🖥️ Acción en Consola / Pizarra:

```
# Demostración práctica recomendada para la Diapositiva 8
docker build -t mi-flask:v1 .
```

💡 Tip de Gestión del Aula:

- *Hacer una pregunta de pausa activa:* "¿Alguien tiene alguna duda sobre FROM: EL PUNTO DE PARTIDA?"

📄 Diapositiva 9: RUN: EJECUTAR COMANDOS DURANTE EL BUILD

Contenido de la PPT:

RUN: EJECUTAR COMANDOS DURANTE EL BUILD

```
1 RUN apt - get update && \
2 apt - get install -y curl && \
3 rm - rf / var / lib / apt / lists /*
```

Limpia archivos temporales (apt lists, pip cache) en el mismo RUN para no arrastrar basura a la imagen final.

Regla de oro

Encadena comandos con && para reducir capas y limpiar cachús en la misma instrucción.

Buena práctica

Ejecuta comandos durante la construcción de la imagen.

Se usa para instalar paquetes, crear carpetas, configurar el sistema.

Cada RUN crea una nueva capa en la imagen.

👤 Guión del Docente (Lo que debes decir a los alumnos):

"Estimados estudiantes, en la **Diapositiva 9** abordamos un punto fundamental: *RUN: EJECUTAR COMANDOS DURANTE EL BUILD*. Revisen los conceptos clave presentados en pantalla y asegúrense de que los estudiantes comprendan cómo se conecta este punto con el laboratorio práctico de la sesión."

Acción en Consola / Pizarra:

```
# Demostración práctica recomendada para la Diapositiva 9
docker build -t mi-flask:v1 .
```

Tip de Gestión del Aula:

- *Hacer una pregunta de pausa activa:* "¿Alguien tiene alguna duda sobre RUN: EJECUTAR COMANDOS DURANTE EL BUILD?"

Diapositiva 10: COPY VS ADD

Contenido de la PPT:

COPY VS ADD

1 # COPY : copia archivos locales

2 COPY requirements . txt .

3 COPY app . py / app /

Prefiere COPY. Usa ADD solo si necesitas descomprimir automáticamente

Regla práctica

Copia archivos o carpetas desde el contexto de build (tu PC) hacia la imagen.

Es la opción recomendada para la mayoría de los casos.

1 # ADD: copia + descomprime + URLs

2 ADD app . tar . gz / app /

3 ADD https :// example . com / file / tmp /

Además de copiar, descomprime

automáticamente archivos .tar.gz.

Puede descargar desde URLs.

¡Solo solo cuando necesites la funcionalidad extra.

Guión del Docente (Lo que debes decir a los alumnos):

"Estimados estudiantes, en la **Diapositiva 10** abordamos un punto fundamental: *COPY VS ADD*. Expliquen aquí el problema clásico en el desarrollo de software. En la laptop del desarrollador todo marcha perfecto porque tiene instaladas librerías específicas o variables locales, pero en el servidor de producción o en la PC del cliente falla. Docker soluciona esto empaquetando todo el entorno junto con la app."

Acción en Consola / Pizarra:

```
# Demostración práctica recomendada para la Diapositiva 10
docker build -t mi-flask:v1 .
```

💡 Tip de Gestión del Aula:

- *Hacer una pregunta de pausa activa:* "¿Alguien tiene alguna duda sobre COPY VS ADD?"

📄 Diapositiva 11: WORKDIR: EL DIRECTORIO DE TRABAJO

Contenido de la PPT:

```
WORKDIR: EL DIRECTORIO DE TRABAJO
1 WORKDIR / app
Prefiere WORKDIR /app; RUN cd ... no
persiste entre instrucciones.
Prefiere WORKDIR
Evita rutas absolutas en cada comando.
Hace el Dockerfile más legible.
Previene errores de archivos en carpetas
equivocadas.
¿Por qué usarlo?
Establece el directorio de trabajo para
las instrucciones siguientes.
Si la carpeta no existe, Docker la crea
automáticamente.
Afecta a RUN, CMD, ENTRYPOINT,
COPY y ADD.
```

🗣️ Guión del Docente (Lo que debes decir a los alumnos):

"Estimados estudiantes, en la **Diapositiva 11** abordamos un punto fundamental: *WORKDIR: EL DIRECTORIO DE TRABAJO*. Expliquen que el Dockerfile es la receta de cocina. Cada instrucción (**FROM**, **WORKDIR**, **COPY**, **RUN**, **CMD**) construye una capa estandarizada e inmutable sobre la cual se monta nuestra aplicación."

👤 Acción en Consola / Pizarra:

```
# Demostración práctica recomendada para la Diapositiva 11
docker build -t mi-flask:v1 .
```

💡 Tip de Gestión del Aula:

- *Hacer una pregunta de pausa activa:* "¿Alguien tiene alguna duda sobre WORKDIR: EL DIRECTORIO DE TRABAJO?"

Diapositiva 12: ENV: VARIABLES DE ENTORNO

Contenido de la PPT:

```
ENV: VARIABLES DE ENTORNO
1 ENV FLASK_ENV = production
2 ENV APP_HOME =/ app
3 ENV PYTHONUNBUFFERED =1
Secretos como contraseñas o tokens
no deben quedar hardcodeados en la
imagen.
Cuándo NO
Configuración por defecto de la app.
Versiones de herramientas.
Rutas y paths estándar.
Cuándo usar ENV
Define variables de entorno disponibles
en tiempo de build y en el contenedor.
Ideal para configuraciones que no
cambian entre entornos.
Se pueden sobrescribir con docker run -e.
```

Guión del Docente (Lo que debes decir a los alumnos):

"Estimados estudiantes, en la **Diapositiva 12** abordamos un punto fundamental: *ENV: VARIABLES DE ENTORNO*. Revisen los conceptos clave presentados en pantalla y asegúrense de que los estudiantes comprendan cómo se conecta este punto con el laboratorio práctico de la sesión."

Acción en Consola / Pizarra:

```
# Demostración práctica recomendada para la Diapositiva 12
docker build -t mi-flask:v1 .
```

Tip de Gestión del Aula:

- *Hacer una pregunta de pausa activa:* "¿Alguien tiene alguna duda sobre ENV: VARIABLES DE ENTORNO?"

Diapositiva 13: EXPOSE: DECLARAR PUERTOS

Contenido de la PPT:

```
EXPOSE: DECLARAR PUERTOS
1 EXPOSE 5000
2 EXPOSE 80/ tcp
EXPOSE: declara (informativo).
docker run -p: publica y mapea el
```

puerto.
EXPOSE vs -p
Documenta qué puertos escucha el contenedor.
No publica el puerto automáticamente.
Sirve como metadata para otros desarrolladores y herramientas.

Guión del Docente (Lo que debes decir a los alumnos):

"Estimados estudiantes, en la **Diapositiva 13** abordamos un punto fundamental: *EXPOSE: DECLARAR PUERTOS*. Recuerden a la clase la regla de oro: NUNCA expongan la base de datos directamente al exterior. Usen redes internas de Docker para que los contenedores se hablen por nombre de servicio (DNS interno) y usen volúmenes nombrados para no perder datos."

Acción en Consola / Pizarra:

```
# Demostración práctica recomendada para la Diapositiva 13
docker build -t mi-flask:v1 .
```

Tip de Gestión del Aula:

- *Hacer una pregunta de pausa activa:* "¿Alguien tiene alguna duda sobre EXPOSE: DECLARAR PUERTOS?"

Diapositiva 14: CMD VS ENTRYPOINT

Contenido de la PPT:

```
CMD VS ENTRYPOINT
1 CMD ["python", "app.py"]
Usa CMD para apps. Usa ENTRYPOINT + CMD para herramientas CLI.
Regla general
Comando por defecto al iniciar el contenedor.
Se puede sobrescribir desde docker run.
Forma exec (array) recomendada sobre forma shell.
1 ENTRYPOINT ["python"]
2 CMD ["app.py"]
Define el ejecutable principal (no se sobrescribe fácilmente).
CMD proporciona argumentos por defecto al ENTRYPOINT.
Util para crear herramientas CLI en contenedores.
```

Guión del Docente (Lo que debes decir a los alumnos):

"Estimados estudiantes, en la **Diapositiva 14** abordamos un punto fundamental: *CMD VS ENTRYPOINT*. Revisen los conceptos clave presentados en pantalla y asegúrense de que los estudiantes comprendan cómo se conecta este punto con el laboratorio práctico de la sesión."

Acción en Consola / Pizarra:

```
# Demostración práctica recomendada para la Diapositiva 14
docker build -t mi-flask:v1 .
```

Tip de Gestión del Aula:

- *Hacer una pregunta de pausa activa:* "¿Alguien tiene alguna duda sobre CMD VS ENTRYPOINT?"

Diapositiva 15: RESUMEN VISUAL DE INSTRUCCIONES

Contenido de la PPT:

```
RESUMEN VISUAL DE INSTRUCCIONES
INSTRUCCIÓN
¿QUÉ HACE?
SOLUCIÓN RÁPIDA
FROM
RUN
COPY
Define la imagen base
Siempre, como primera línea
Ejecuta comandos en el build
Copia archivos locales a la imagen
WORKDIR
Establece el directorio de trabajo
Instalar dependencias, scripts de setup
Mover código y archivos del proyecto
Al inicio y cuando cambies de contexto
ENV
EXPOSE
CMD
Define variables de entorno
Declara puertos de escucha
Comando por defecto del contenedor
Configuraciones por defecto
Documentar qué puertos usa la app
Iniciar la aplicación
```

Guión del Docente (Lo que debes decir a los alumnos):

"Estimados estudiantes, en la **Diapositiva 15** abordamos un punto fundamental: *RESUMEN VISUAL DE INSTRUCCIONES*. Expliquen que el Dockerfile es la receta de cocina. Cada instrucción (*FROM*, *WORKDIR*,

COPY, RUN, CMD) construye una capa estandarizada e inmutable sobre la cual se monta nuestra aplicación."

Acción en Consola / Pizarra:

```
# Demostración práctica recomendada para la Diapositiva 15
docker build -t mi-flask:v1 .
```

Tip de Gestión del Aula:

- *Hacer una pregunta de pausa activa:* "¿Alguien tiene alguna duda sobre RESUMEN VISUAL DE INSTRUCCIONES?"

Diapositiva 16: Capas, cachÚ y

Contenido de la PPT:

Capas, cachÚ y
buenas prðcticas

Guión del Docente (Lo que debes decir a los alumnos):

"Estimados estudiantes, en la **Diapositiva 16** abordamos un punto fundamental: *Capas, cachÚ y*. Revisen los conceptos clave presentados en pantalla y asegúrense de que los estudiantes comprendan cómo se conecta este punto con el laboratorio práctico de la sesión."

 **Acción en Consola / Pizarra:** *Escribir en la pizarra o proyectar el diagrama del flujo correspondiente.*

Tip de Gestión del Aula:

- *Hacer una pregunta de pausa activa:* "¿Alguien tiene alguna duda sobre Capas, cachÚ y?"

Diapositiva 17: ¿ CĖMO CONSTRUYE DOCKER UNA IMAGEN?

Contenido de la PPT:

¿ CĖMO CONSTRUYE DOCKER UNA IMAGEN?
Las capas son inmutables. Docker reutiliza las que no cambiaron mediante cachÚ.
Cada instruccióñ del Dockerfile genera una capa

Guión del Docente (Lo que debes decir a los alumnos):

"Estimados estudiantes, en la **Diapositiva 17** abordamos un punto fundamental: ¿ CĖMO CONSTRUYE DOCKER UNA IMAGEN?. Expliquen que el Dockerfile es la receta de cocina. Cada instrucción (**FROM**,

WORKDIR, COPY, RUN, CMD) construye una capa estandarizada e inmutable sobre la cual se monta nuestra aplicación."

Acción en Consola / Pizarra:

```
# Demostración práctica recomendada para la Diapositiva 17
docker build -t mi-flask:v1 .
```

Tip de Gestión del Aula:

- *Hacer una pregunta de pausa activa:* "¿Alguien tiene alguna duda sobre ¿CÓMO CONSTRUYE DOCKER UNA IMAGEN??"

Diapositiva 18: Docker compara cada instrucción con la cachú de builds

Contenido de la PPT:

Docker compara cada instrucción con la cachú de builds anteriores.
Si la instrucción y sus dependencias no cambiaron, reutiliza la capa.
Si una capa cambia, todas las capas siguientes se reconstruyen.
Pon primero lo que cambia menos (instalación de dependencias) y al final lo que cambia más (código de la app).
Regla de oro del orden
La cachú de Docker: cómo funciona

Guión del Docente (Lo que debes decir a los alumnos):

"Estimados estudiantes, en la **Diapositiva 18** abordamos un punto fundamental: *Docker compara cada instrucción con la cachú de builds*. Expliquen aquí el problema clásico en el desarrollo de software. En la laptop del desarrollador todo marcha perfecto porque tiene instaladas librerías específicas o variables locales, pero en el servidor de producción o en la PC del cliente falla. Docker soluciona esto empaquetando todo el entorno junto con la app."

Acción en Consola / Pizarra:

```
# Demostración práctica recomendada para la Diapositiva 18
docker build -t mi-flask:v1 .
```

💡 Tip de Gestión del Aula:

- *Hacer una pregunta de pausa activa:* "¿Alguien tiene alguna duda sobre Docker compara cada instrucción con la caché de builds?"

📄 Diapositiva 19: EJEMPLO: DOCKERFILE OPTIMIZADO

Contenido de la PPT:

EJEMPLO: DOCKERFILE OPTIMIZADO

```
1 FROM python :3.12
3 COPY . / app # TODO cambia
4 WORKDIR / app
6 RUN pip install \ # Siempre
7 -r requirements . txt
9 CMD [" python "," app .py"]
Cachea dependencias; cambios en app.py no reinstalan paquetes.
¿Por qué es más rápido?
Mal orden (lento)
1 FROM python :3.12 - slim
3 WORKDIR / app
5 # Dependencias primero
6 COPY requirements . txt .
7 RUN pip install --no-cache-dir \
8 -r requirements . txt
10 # Código al final
11 COPY . .
13 CMD [" python "," app .py"]
Buen orden (rápido)
```

👤 Guión del Docente (Lo que debes decir a los alumnos):

"Estimados estudiantes, en la **Diapositiva 19** abordamos un punto fundamental: *EJEMPLO: DOCKERFILE OPTIMIZADO*. Expliquen que el Dockerfile es la receta de cocina. Cada instrucción (**FROM**, **WORKDIR**, **COPY**, **RUN**, **CMD**) construye una capa estandarizada e inmutable sobre la cual se monta nuestra aplicación."

🖥️ Acción en Consola / Pizarra:

```
# Demostración práctica recomendada para la Diapositiva 19
docker build -t mi-flask:v1 .
```

💡 Tip de Gestión del Aula:

- *Hacer una pregunta de pausa activa:* "¿Alguien tiene alguna duda sobre EJEMPLO: DOCKERFILE OPTIMIZADO?"

Diapositiva 20: OTRAS BUENAS PRÁCTICAS

Contenido de la PPT:

```
OTRAS BUENAS PRÁCTICAS
1 # Mal : imagen generica y pesada
2 FROM python
4 # Bien : version fija y liviana
5 FROM python :3.12 - slim
1. Usa imágenes base específicas
1 RUN pip install -r requirements . txt && \
2 rm - rf / root /. cache / pip
3. Limpia en el mismo RUN
1 RUN apt - get update && \
2 apt - get install -y --no - install - recommends \
3 curl && \
4 rm - rf / var / lib / apt / lists /*
2. No instales paquetes innecesarios
1 # Stage 1: build
2 FROM golang :1.21 AS builder
3 RUN go build -o app
5 # Stage 2: runtime ( imagen final peque ña)
6 FROM alpine :3.19
7 COPY -- from = builder / app .
8 CMD ["/ app "]
4. Usa multistage builds
```

Guión del Docente (Lo que debes decir a los alumnos):

"Estimados estudiantes, en la **Diapositiva 20** abordamos un punto fundamental: *OTRAS BUENAS PRÁCTICAS*. Expliquen que el Dockerfile es la receta de cocina. Cada instrucción (**FROM**, **WORKDIR**, **COPY**, **RUN**, **CMD**) construye una capa estandarizada e inmutable sobre la cual se monta nuestra aplicación."

Acción en Consola / Pizarra:

```
# Demostración práctica recomendada para la Diapositiva 20
docker build -t mi-flask:v1 .
```

Tip de Gestión del Aula:

- *Hacer una pregunta de pausa activa:* "¿Alguien tiene alguna duda sobre OTRAS BUENAS PRÁCTICAS?"

Diapositiva 21: MULTISTAGE BUILD: EL PATRÓN PROFESIONAL

Contenido de la PPT:

MULTISTAGE BUILD: EL PATR N PROFESIONAL

La imagen final solo contiene lo necesario para ejecutar. Las herramientas de compilaci n

quedan en la etapa de build y no contaminan la imagen de producci n.

Ventaja

  Gui n del Docente (Lo que debes decir a los alumnos):

"Estimados estudiantes, en la **Diapositiva 21** abordamos un punto fundamental: *MULTISTAGE BUILD: EL PATR N PROFESIONAL*. Revisen los conceptos clave presentados en pantalla y aseg rense de que los estudiantes comprendan c mo se conecta este punto con el laboratorio pr ctico de la sesi n."

  Acci n en Consola / Pizarra:

```
# Demostraci n pr ctica recomendada para la Diapositiva 21
docker build -t mi-flask:v1 .
```

  Tip de gesti n del aula:

- *Hacer una pregunta de pausa activa:* " Alguien tiene alguna duda sobre MULTISTAGE BUILD: EL PATR N PROFESIONAL?"

  Diapositiva 22: .dockerignore e**Contenido de la PPT:**

```
.dockerignore e
im genes base livianas
```

  Gui n del Docente (Lo que debes decir a los alumnos):

"Estimados estudiantes, en la **Diapositiva 22** abordamos un punto fundamental: *.dockerignore e*. Revisen los conceptos clave presentados en pantalla y aseg rense de que los estudiantes comprendan c mo se conecta este punto con el laboratorio pr ctico de la sesi n."

  Acci n en Consola / Pizarra:

```
# Demostraci n pr ctica recomendada para la Diapositiva 22
docker build -t mi-flask:v1 .
```

  Tip de gesti n del aula:

- *Hacer una pregunta de pausa activa:* " Alguien tiene alguna duda sobre .dockerignore e?"

Diapositiva 23: .DOCKERIGNORE: QUÉ ES Y PARA QUÉ SIRVE

Contenido de la PPT:

```
.DOCKERIGNORE: QUÉ ES Y PARA QUÉ SIRVE
# .dockerignore
__pycache__
*.pyc
*.pyo
.git
.env
*.log
node_modules
venv/
.venv/
.vscode/
*.md
Dockerfile
docker-compose.yml
Incluye .dockerignore; evita enviar node_modules, .git y credenciales.
Regla
Similar a .gitignore: archivos y carpetas
que no se envían al contexto de build.
Reduce el tamaño del contexto y
acelera el build.
Evita que archivos sensibles (.env,
claves) terminen en la imagen.
```

Guión del Docente (Lo que debes decir a los alumnos):

"Estimados estudiantes, en la **Diapositiva 23** abordamos un punto fundamental: **.DOCKERIGNORE: QUÉ ES Y PARA QUÉ SIRVE**. Expliquen que el Dockerfile es la receta de cocina. Cada instrucción (**FROM**, **WORKDIR**, **COPY**, **RUN**, **CMD**) construye una capa estandarizada e inmutable sobre la cual se monta nuestra aplicación."

Acción en Consola / Pizarra:

```
# Demostración práctica recomendada para la Diapositiva 23
docker build -t mi-flask:v1 .
```

Tip de Gestión del Aula:

- *Hacer una pregunta de pausa activa:* "¿Alguien tiene alguna duda sobre **.DOCKERIGNORE: QUÉ ES Y PARA QUÉ SIRVE?**"

Diapositiva 24: IMÁGENES BASE: SLIM VS ALPINE

Contenido de la PPT:

IM⁺GENES BASE: SLIM VS ALPINE

Slim: buena relación tamaño/compatibilidad.

Alpine: máxima reducción, pero puede tener problemas con paquetes que dependen de glibc.

Para empezar, slim es la opción más segura.

¿Cuál elegir?

Guión del Docente (Lo que debes decir a los alumnos):

"Estimados estudiantes, en la **Diapositiva 24** abordamos un punto fundamental: *IM⁺GENES BASE: SLIM VS ALPINE*. Expliquen aquí el problema clásico en el desarrollo de software. En la laptop del desarrollador todo marcha perfecto porque tiene instaladas librerías específicas o variables locales, pero en el servidor de producción o en la PC del cliente falla. Docker soluciona esto empaquetando todo el entorno junto con la app."

 **Acción en Consola / Pizarra:** *Escribir en la pizarra o proyectar el diagrama del flujo correspondiente.*

Tip de Gestión del Aula:

- *Hacer una pregunta de pausa activa:* "¿Alguien tiene alguna duda sobre IM⁺GENES BASE: SLIM VS ALPINE?"

Diapositiva 25: Publicación en

Contenido de la PPT:

Publicación en
Docker Hub

Guión del Docente (Lo que debes decir a los alumnos):

"Estimados estudiantes, en la **Diapositiva 25** abordamos un punto fundamental: *Publicación en*. Revisen los conceptos clave presentados en pantalla y asegúrense de que los estudiantes comprendan cómo se conecta este punto con el laboratorio práctico de la sesión."

 **Acción en Consola / Pizarra:**

```
# Demostración práctica recomendada para la Diapositiva 25
docker build -t mi-flask:v1 .
```

Tip de Gestión del Aula:

- *Hacer una pregunta de pausa activa:* "¿Alguien tiene alguna duda sobre Publicación en?"

Diapositiva 26: FLUJO DE PUBLICACIÓN

Contenido de la PPT:

FLUJO DE PUBLICACIÖN

1. Construyes tu imagen.
 2. Le asignas un tag.
 3. La publicas en un registro.
 4. Cualquiera puede descargarla y usarla.
- El ciclo completo

👤 Guión del Docente (Lo que debes decir a los alumnos):

"Estimados estudiantes, en la **Diapositiva 26** abordamos un punto fundamental: *FLUJO DE PUBLICACIÖN*. Revisen los conceptos clave presentados en pantalla y asegúrense de que los estudiantes comprendan cómo se conecta este punto con el laboratorio práctico de la sesión."

🏠 Acción en Consola / Pizarra: *Escribir en la pizarra o proyectar el diagrama del flujo correspondiente.*

💡 Tip de Gestión del Aula:

- *Hacer una pregunta de pausa activa: "¿Alguien tiene alguna duda sobre FLUJO DE PUBLICACIÖN?"*

📄 Diapositiva 27: LOGIN Y TAGGING**Contenido de la PPT:**

LOGIN Y TAGGING

```
docker login
```

```
# Usuario: tu-usuario
```

```
# Password: tu-token
```

1. Iniciar sesiön en Docker Hub

Convenciön de tags

```
# Formato:
```

```
# usuario/imagen:tag
```

```
docker tag mi-flask:v1 \
```

```
tony/mi-flask:v1
```

2. Etiquetar la imagen

Importante

v1, v2, v3: versiones especÝficas.

latest: la mßs reciente (se actualiza manualmente).

dev, staging, prod: por entorno.

2026-05-31: por fecha de build.

latest no es automßtico: t. decides quÜ imagen lo lleva.

👤 Guión del Docente (Lo que debes decir a los alumnos):

"Estimados estudiantes, en la **Diapositiva 27** abordamos un punto fundamental: *LOGIN Y TAGGING*. Revisen los conceptos clave presentados en pantalla y asegúrense de que los estudiantes comprendan

cómo se conecta este punto con el laboratorio práctico de la sesión."

Acción en Consola / Pizarra:

```
# Demostración práctica recomendada para la Diapositiva 27
docker build -t mi-flask:v1 .
```

Tip de Gestión del Aula:

- *Hacer una pregunta de pausa activa: "¿Alguien tiene alguna duda sobre LOGIN Y TAGGING?"*

Diapositiva 28: PUSH Y PULL

Contenido de la PPT:

```
PUSH Y PULL
docker push tony/mi-flask:v1
docker push tony/mi-flask:latest
3. Publicar la imagen
4. Descargar y ejecutar
¿Pública o privada?
Docker Hub permite 1 repo privado gratis.
GHCR es alternativa para repos privados.
La imagen queda disponible en Docker Hub
según la visibilidad del repositorio.
docker pull tony/mi-flask:v1
docker run -d -p 5000:5000 \
tony/mi-flask:v1
```

Guión del Docente (Lo que debes decir a los alumnos):

"Estimados estudiantes, en la **Diapositiva 28** abordamos un punto fundamental: *PUSH Y PULL*. Revisen los conceptos clave presentados en pantalla y asegúrense de que los estudiantes comprendan cómo se conecta este punto con el laboratorio práctico de la sesión."

Acción en Consola / Pizarra:

```
# Demostración práctica recomendada para la Diapositiva 28
docker build -t mi-flask:v1 .
```

Tip de Gestión del Aula:

- *Hacer una pregunta de pausa activa: "¿Alguien tiene alguna duda sobre PUSH Y PULL?"*

Diapositiva 29: Laboratorio

Contenido de la PPT:

Laboratorio
práctico

👤 Guión del Docente (Lo que debes decir a los alumnos):

"Estimados estudiantes, en la **Diapositiva 29** abordamos un punto fundamental: *Laboratorio*. Revisen los conceptos clave presentados en pantalla y asegúrense de que los estudiantes comprendan cómo se conecta este punto con el laboratorio práctico de la sesión."

🏠 Acción en Consola / Pizarra: *Escribir en la pizarra o proyectar el diagrama del flujo correspondiente.*

💡 Tip de Gestión del Aula:

- *Hacer una pregunta de pausa activa: "¿Alguien tiene alguna duda sobre Laboratorio?"*

📄 Diapositiva 30: LABORATORIO: APP FLASK OPTIMIZADA

Contenido de la PPT:

LABORATORIO: APP FLASK OPTIMIZADA
Partiendo del proyecto Flask de la sesión 1, optimizar el Dockerfile con buenas prácticas,
construir la imagen aprovechando cachú y publicarla en Docker Hub.
Objetivo

👤 Guión del Docente (Lo que debes decir a los alumnos):

"Estimados estudiantes, en la **Diapositiva 30** abordamos un punto fundamental: *LABORATORIO: APP FLASK OPTIMIZADA*. Expliquen que el Dockerfile es la receta de cocina. Cada instrucción (**FROM**, **WORKDIR**, **COPY**, **RUN**, **CMD**) construye una capa estandarizada e inmutable sobre la cual se monta nuestra aplicación."

🏠 Acción en Consola / Pizarra:

```
# Demostración práctica recomendada para la Diapositiva 30  
docker build -t mi-flask:v1 .
```

💡 Tip de Gestión del Aula:

- *Hacer una pregunta de pausa activa: "¿Alguien tiene alguna duda sobre LABORATORIO: APP FLASK OPTIMIZADA?"*

📄 Diapositiva 31: DOCKERFILE OPTIMIZADO PARA EL LABORATORIO

Contenido de la PPT:

```
DOCKERFILE OPTIMIZADO PARA EL LABORATORIO
1 # Etapa 1: builder (si necesitas compilar )
2 FROM python :3.12 - slim AS builder
3 WORKDIR / app
4 COPY requirements . txt .
5 RUN pip install --no - cache - dir -r requirements . txt
7 # Etapa 2: imagen final ( runtime )
8 FROM python :3.12 - slim
9 WORKDIR / app
10 # Copiar dependencias ya instaladas desde el builder
11 COPY -- from = builder / usr / local / lib / python3 .12/ site - packages \
12 / usr / local / lib / python3 .12/ site - packages
13 COPY -- from = builder / usr / local / bin / usr / local / bin 14 COPY app .
py . 15 EXPOSE 5000
16 CMD [" python ", " app .py"]
```

Guión del Docente (Lo que debes decir a los alumnos):

"Estimados estudiantes, en la **Diapositiva 31** abordamos un punto fundamental: *DOCKERFILE OPTIMIZADO PARA EL LABORATORIO*. Expliquen que el Dockerfile es la receta de cocina. Cada instrucción (**FROM**, **WORKDIR**, **COPY**, **RUN**, **CMD**) construye una capa estandarizada e inmutable sobre la cual se monta nuestra aplicación."

Acción en Consola / Pizarra:

```
# Demostración práctica recomendada para la Diapositiva 31
docker build -t mi-flask:v1 .
```

Tip de Gestión del Aula:

- *Hacer una pregunta de pausa activa:* "¿Alguien tiene alguna duda sobre DOCKERFILE OPTIMIZADO PARA EL LABORATORIO?"

Diapositiva 32: COMANDOS DEL LABORATORIO

Contenido de la PPT:

```
COMANDOS DEL LABORATORIO
# 1. Construir la imagen
docker build -t mi-flask:v2 .
# 2. Ver las capas generadas
docker history mi-flask:v2
# 3. Ejecutar y probar
docker run -d --name flask-v2 \
-p 5000:5000 mi-flask:v2
```

```
curl http://localhost:5000
# 4. Login en Docker Hub
docker login
# 5. Etiquetar
docker tag mi-flask:v2 \ tony/mi-flask:v2
# 6. Publicar
docker push tony/mi-flask:v2
# 7. Verificar en:
# hub.docker.com/r/tony/mi-flask
```

Guión del Docente (Lo que debes decir a los alumnos):

"Estimados estudiantes, en la **Diapositiva 32** abordamos un punto fundamental: *COMANDOS DEL LABORATORIO*. Revisen los conceptos clave presentados en pantalla y asegúrense de que los estudiantes comprendan cómo se conecta este punto con el laboratorio práctico de la sesión."

Acción en Consola / Pizarra:

```
# Demostración práctica recomendada para la Diapositiva 32
docker build -t mi-flask:v1 .
```

Tip de Gestión del Aula:

- *Hacer una pregunta de pausa activa:* "¿Alguien tiene alguna duda sobre COMANDOS DEL LABORATORIO?"

Diapositiva 33: ERRORES FRECUENTES EN ESTA SESIÓN

Contenido de la PPT:

```
ERRORES FRECUENTES EN ESTA SESIÓN
ERROR
CAUSA PROBABLE
SOLUCIÓN RÁPIDA
Build lento siempre
Imagen muy pesada
CachÚ no funciona
Mal orden de capas (COPY . . antes de RUN)
Mover dependencias antes del código
Usar imagen base :latest completa
Cambiar un archivo antes de lo esperado
docker push rechazado
No hay login o tag mal escrito
Cambiar a :slim o :alpine
Revisar orden: lo estable primero
docker login y verificar usuario/imagen:tag
Archivos .env en la imagen
Error no space left
```

```
Falta .dockerignore
Imágenes acumuladas en disco
Crear .dockerignore y reconstruir
docker system prune -a
```

Guión del Docente (Lo que debes decir a los alumnos):

"Estimados estudiantes, en la **Diapositiva 33** abordamos un punto fundamental: *ERRORES FRECUENTES EN ESTA SESIÓN*. Expliquen aquí el problema clásico en el desarrollo de software. En la laptop del desarrollador todo marcha perfecto porque tiene instaladas librerías específicas o variables locales, pero en el servidor de producción o en la PC del cliente falla. Docker soluciona esto empaquetando todo el entorno junto con la app."

Acción en Consola / Pizarra:

```
# Demostración práctica recomendada para la Diapositiva 33
docker build -t mi-flask:v1 .
```

Tip de Gestión del Aula:

- *Hacer una pregunta de pausa activa:* "¿Alguien tiene alguna duda sobre ERRORES FRECUENTES EN ESTA SESIÓN?"

Diapositiva 34: CHECKLIST DE APRENDIZAJE ù SESIÓN 2

Contenido de la PPT:

```
CHECKLIST DE APRENDIZAJE ù SESIÓN 2
1.Puedo escribir un Dockerfile con FROM, RUN, COPY, WORKDIR, ENV,
EXPOSE y CMD.
2.Puedo explicar la diferencia entre COPY y ADD, CMD y ENTRYPOINT.
3.Puedo explicar qué es una capa y cómo afecta el orden de las
instrucciones.
4.Puedo optimizar un Dockerfile para aprovechar la cachú.
5.Puedo usar .dockerignore para excluir archivos del contexto de build.
6.Puedo elegir entre imagen base completa, slim y alpine.
7.Puedo publicar una imagen en Docker Hub con tagging y versionado.
8.Puedo descargar y ejecutar una imagen desde Docker Hub.
```

Guión del Docente (Lo que debes decir a los alumnos):

"Estimados estudiantes, en la **Diapositiva 34** abordamos un punto fundamental: *CHECKLIST DE APRENDIZAJE ù SESIÓN 2*. Expliquen que el Dockerfile es la receta de cocina. Cada instrucción (**FROM**, **WORKDIR**, **COPY**, **RUN**, **CMD**) construye una capa estandarizada e inmutable sobre la cual se monta nuestra aplicación."

 Acción en Consola / Pizarra:

```
# Demostración práctica recomendada para la Diapositiva 34
docker build -t mi-flask:v1 .
```

 Tip de Gestión del Aula:

- *Hacer una pregunta de pausa activa:* "¿Alguien tiene alguna duda sobre CHECKLIST DE APRENDIZAJE ù SESIÖN 2?"
-